

A BRIGHTER
TOMORROW
TOGETHER

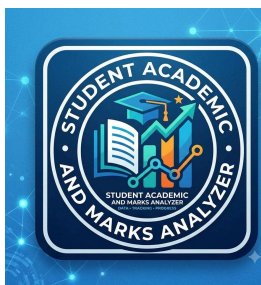


VIT BHOPAL
UNIVERSITY
INDIA'S GLOBAL UNIVERSITY

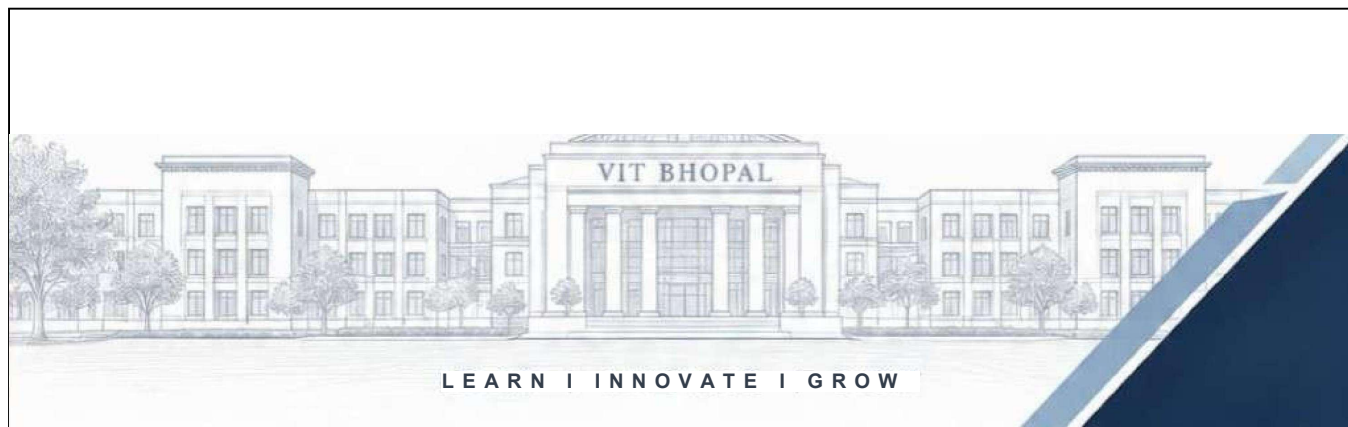
———— # BUILD YOUR OWN PROJECT (BYOP) ————

STUDENT ACADEMIC AND MARKS
.....ANALYZER.....

A Python-Based Application for Student's
Academic Performance



Name:Nalin Pandita
Branch:Computer Science Core
Registration number:26BCE11488



PROJECT REPORT

SAMA: STUDENT ACADEMIC & MARKS ANALYZER

1. INTRODUCTION

SAMA (Student Academic & Marks Analyzer) is a small Python command-line program made to help students get a quick summary of their marks. It can be used for a term, test, or any set of subjects.

The program first asks for the student's name, number of subjects, maximum marks, subject names, and marks. After taking these inputs, it works out the total, average, percentage, and the subjects with the highest and lowest marks.

Since this is a beginner-level project, I kept the program focused on basic Python concepts. It uses variables, input and output, lists, for loops, if statements, and built-in functions to solve a simple academic task.

2. PROBLEM STATEMENT

Working out results by hand can get confusing, especially when there are several subjects and calculations involved. It is easy to make a mistake in the total, average, percentage, or when checking which subject scored the most or least. A spreadsheet can do this too, but it can be more than what is needed for a quick result.

SAMA keeps the process simple by doing these calculations through the terminal. The user enters the marks once, and the program immediately shows the main results without needing another application or a complicated setup.

3. FUNCTIONAL REQUIREMENTS

Requirement	Feature	Description
FR-01	Student Name Input	The program shall ask for and store the student's name.
FR-02	Exam Setup	The program shall take the total number of subjects and maximum marks per exam as inputs.
FR-03	Marks Entry	The program shall prompt the user to enter each subject name and the corresponding marks obtained.
FR-04	Total & Average	The program shall calculate total marks using <code>sum()</code> and average marks by dividing total marks by the number of subjects.
FR-05	Percentage	The program shall calculate percentage using the formula: $(\text{average} / \text{maximum marks}) * 100$.
FR-06	Highest & Lowest	The program shall find the maximum score using <code>max()</code> , minimum score using <code>min()</code> , and identify the corresponding subjects using a loop.
FR-07	Report Output	The program shall print a clear summary report on the screen using <code>print()</code> statements.

4. NON-FUNCTIONAL REQUIREMENTS

Simplicity

SAMA uses normal terminal prompts, so there is no complicated interface to learn. It can be run directly from a Python terminal.

Environment Compatibility

The program is meant to run in a normal Python 3.x environment on Windows, macOS, or Linux.

Zero External Dependencies

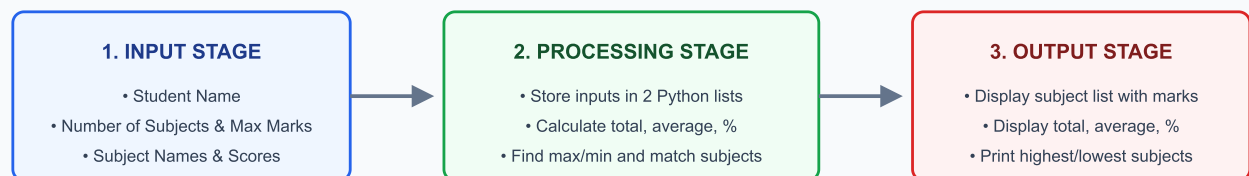
SAMA only uses Python's built-in features, so there is no need to install extra packages with pip.

Readability

The code is arranged in a simple step-by-step way and uses clear variable names, so it is easier for a beginner to read and modify.

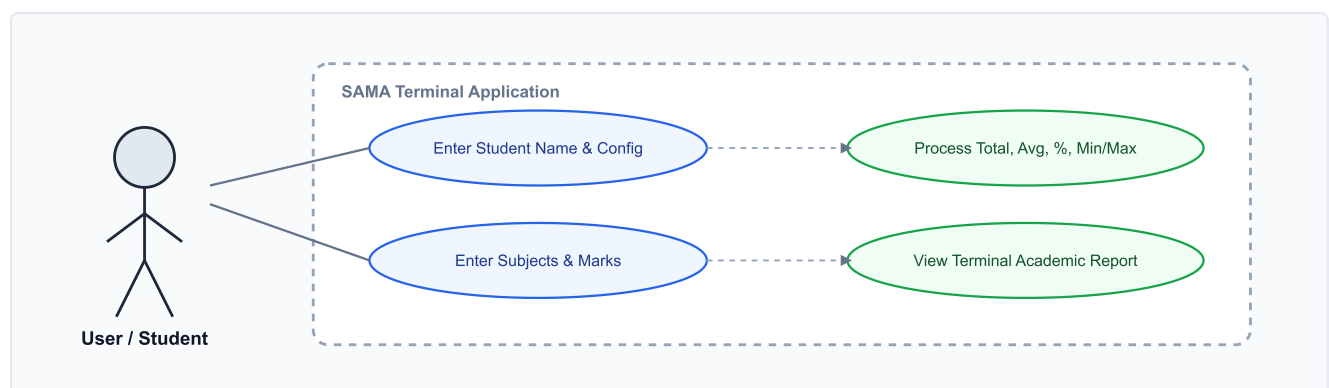
5. SYSTEM ARCHITECTURE

SAMA uses a simple Input-Process-Output (IPO) flow. Everything happens in order inside the SAMA.py file, starting with user input and ending with the report.

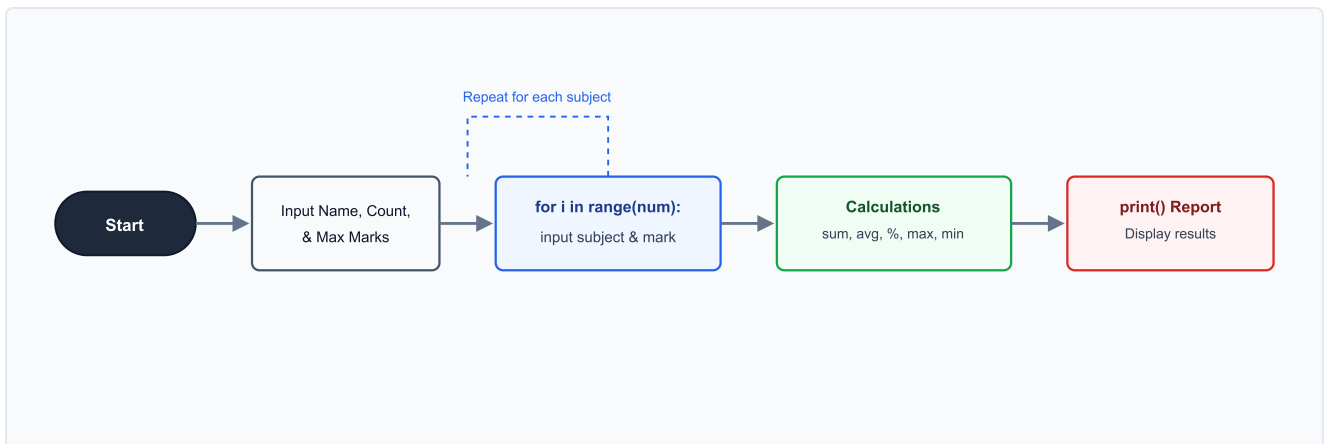


6. DESIGN DIAGRAMS

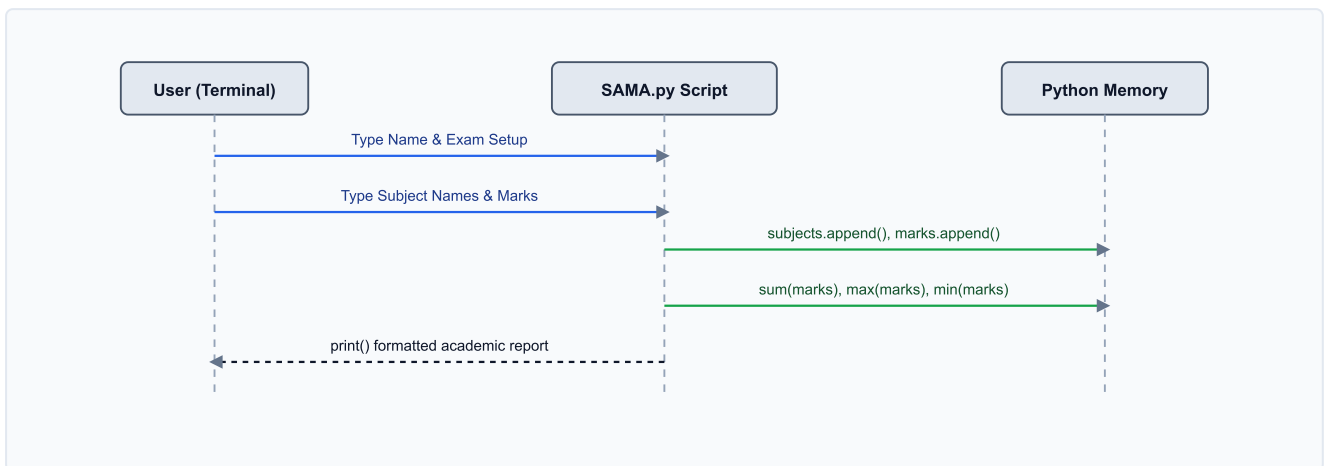
6.1 Use Case Diagram



6.2 Workflow Diagram

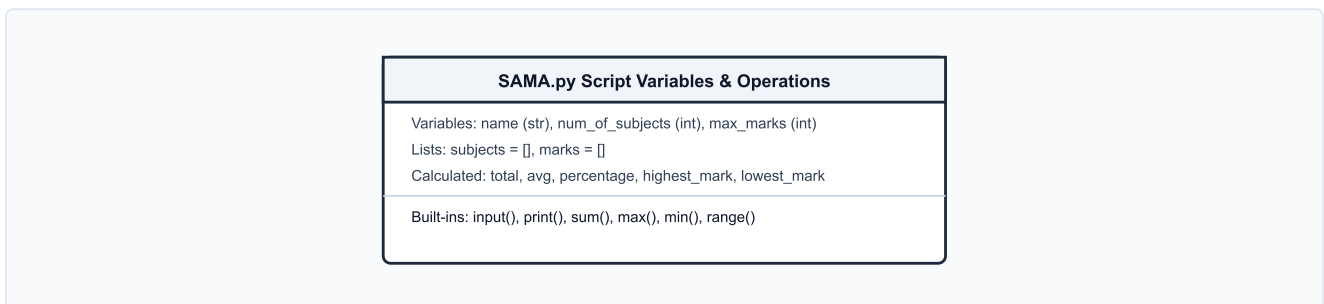


6.3 Sequence Diagram



6.4 Component Structure Diagram

SAMA is a procedural Python script, so the main parts are just its variables, lists, calculated values, and the built-in functions used to process them.



6.5 Entity-Relationship (ER) Diagram Notice

An ER diagram is not really needed for SAMA because the project does not use a database or SQL tables. The entered student and subject information only stays in Python memory while the program is running.

7. DESIGN DECISIONS & RATIONALE

1. Parallel Lists

I used two separate lists: one for subject names and one for marks. The same index in both lists refers to the same subject and its mark.

2. Converting Inputs to float

The marks are converted to float so the program can accept decimal values such as 42.5. This is useful because marks are not always whole numbers.

3. Command-Line Interface

Keeping the interface in the terminal makes the project small and easy to run. It also avoids the need for extra libraries or a graphical interface.

4. Built-in Functions

Using `sum()`, `max()`, and `min()` keeps the main calculations short and easy to read.

8. IMPLEMENTATION DETAILS

Data Input and List Storage

A for loop repeats once for each subject. In every iteration, the program asks for the subject name and its mark, then stores both values in their respective lists.

```
for i in range(num_of_subjects):
    subject = input("Enter subject name: ")
    mark = float(input("Enter marks for subject: "))
    subjects.append(subject)
    marks.append(mark)
```

Calculations

```
total = sum(marks)
avg = total / num_of_subjects
percentage = (avg / max_marks) * 100
highest_mark = max(marks)
lowest_mark = min(marks)
```

Matching Highest and Lowest Subjects

Another loop checks the marks against the highest and lowest values. This is how the program connects those values back to the names of the subjects that got them.

```
for i in range(num_of_subjects):
    if marks[i] == highest_mark:
        most_scoring_subject = subjects[i]
    if marks[i] == lowest_mark:
        lowest_scoring_subject = subjects[i]
```

Additional Technical Notes

Current Non-Functional Details

A few extra points are worth mentioning about the current version. The code is fairly easy to maintain because the logic is small and straightforward. It does not save data after the program closes, and everything is handled through the terminal. Input validation is also limited at the moment, so some invalid values can cause errors.

Current Limitations

- Non-numeric input for a numeric field can raise an exception because the program directly uses `int()` and `float()`.
- Entering zero subjects can cause a division-by-zero error when calculating the average.
- Entering zero as the maximum marks can cause a division-by-zero error when calculating the percentage.
- When multiple subjects share the highest or lowest mark, the later matching subject overwrites the earlier one, so only one subject is displayed.

Testing Beyond the Normal Case

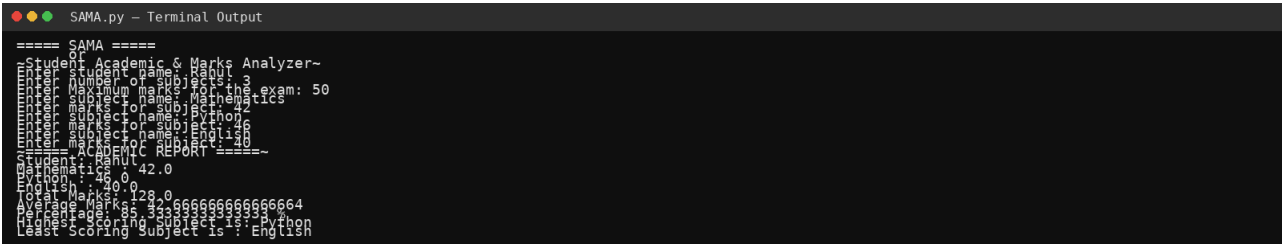
Case	Observed behaviour
Tied highest / lowest marks	The program completes, but the later matching subject replaces the earlier result.
Zero subjects	Average calculation reaches division by zero; this is a documented limitation.
Maximum marks = 0	Percentage calculation reaches division by zero; this is a documented limitation.
Invalid numeric input	Direct <code>int()/float()</code> conversion raises an exception; validation is not yet implemented.

Design Trade-off

I intentionally kept the project small: one Python file, two lists, built-in functions, and terminal output. This makes the code easier to understand while leaving things like validation, file storage, charts, and deeper analysis for possible future versions.

9. SCREENSHOTS / RESULTS

Actual terminal execution of SAMA.py



```
===== SAMA =====
SAMA: Student Academic & Marks Analyzer~
Enter the number of subjects: 50
Enter the marks of the subjects:
Enter the subject name: Rahul
Enter the marks: 42.0
Enter the subject name: Python
Enter the marks: 46.0
Enter the subject name: English
Enter the marks: 40.0
Enter the subject name: Priya
Enter the marks: 82.5
Enter the subject name: Chemistry
Enter the marks: 91.0
Enter the subject name: Amit
Enter the marks: 95.0
Enter the subject name: Computer Science
Enter the marks: 95.0
Enter the subject name: Siddharth
Enter the marks: 50.0
Enter the subject name: Physics
Enter the marks: 50.0
Enter the subject name: English
Enter the marks: 35.0
Enter the subject name: Neha
Enter the marks: 90.0
Enter the subject name: Biology
Enter the marks: 40.0
Enter the subject name: Physics
Enter the marks: 40.0
=====
Total: 128.0, Avg: 42.67, Percentage: 85.33%. Highest: Python, Lowest: English.
Calculated accurately.
```

10. TESTING APPROACH

I tested the program by running SAMA.py with normal inputs as well as a few edge cases. The results below were taken from actual runs of the program so that the table reflects what the current code really does.

Test ID	Scenario	Input Details	Observed Behavior & Result
TC-01	Standard Multi-Subject	Rahul 3 subjects, Max 50 Math: 42, Python: 46, English: 40	Total: 128.0, Avg: 42.67, Percentage: 85.33%. Highest: Python, Lowest: English. Calculated accurately.
TC-02	Decimal Marks Entry	Priya 2 subjects, Max 100 Physics: 82.5, Chemistry: 91.0	Total: 173.5, Avg: 86.75, Percentage: 86.75%. Highest: Chemistry, Lowest: Physics. Handled floating-point inputs cleanly.
TC-03	Single Subject Entry	Amit 1 subject, Max 100 Computer Science: 95	Total: 95.0, Avg: 95.0, Percentage: 95.0%. Both highest and lowest scoring subject evaluated to "Computer Science".
TC-04	Tied Highest Score	Siddharth 3 subjects, Max 50 Math: 50, Physics: 50, English: 35	Highest mark evaluated to 50. In the loop, <code>most_scoring_subject</code> was set to Math first, then overwritten by Physics (the last match).
TC-05	Tied Lowest Score	Neha 3 subjects, Max 100 Biology: 90, Chemistry: 40, Physics: 40	Lowest mark evaluated to 40. In the loop, <code>lowest_scoring_subject</code> was set to Chemistry first, then overwritten by Physics.

11. CHALLENGES FACED

- List Index Matching: Getting used to using the same loop index `i` for both lists was one of the important parts, since `subjects[i]` and `marks[i]` need to refer to the same subject.
- Data Type Conversion: I had to make sure the values coming from `input()` were converted from strings to `int` or `float` before using them in calculations.
- Handling Tied Scores: Testing showed that if two subjects have the same highest or lowest mark, the variable gets overwritten and the last matching subject is displayed.
- Finding Subject Names for Highest/Lowest Marks: Another useful part was figuring out how to take the result of `max(marks)` or `min(marks)` and match it back to the correct subject using a loop and an `if` condition.

12. LEARNINGS & KEY TAKEAWAYS

Variables & User Input: Taking information from the user and converting input strings into numbers with `int()` and `float()`.
Python Lists: Using lists to keep the subjects and marks together, and adding new values with `.append()`.
Control Structures: Using for loops to repeat the input process and if statements to check conditions.
Built-in Functions: Using `sum()`, `max()`, and `min()` for the main calculations.
Program Flow: Putting the whole process together, from collecting data to processing it and finally printing the report.

13. FUTURE ENHANCEMENTS

Better Tie Handling

Instead of keeping only one matching subject, the program could store all subjects that share the highest or lowest mark.

Grade Calculation

The program could assign letter grades such as A, B, or C automatically based on the percentage.

File Storage

The generated report could be saved as a text or CSV file so it can be opened again later.

Data Visualization

A future version could also show the marks visually with bar charts, for example using Matplotlib.

14. REFERENCES

1. Python Documentation — Built-in Functions: <https://docs.python.org/3/library/functions.html>
2. Python Documentation — More on Lists: <https://docs.python.org/3/tutorial/datastructures.html>